Hubitat Dashboard — Complete User Guide

> **Who is this guide for?** Anyone who wants to run the Hubitat Dashboard — web or Android — and has never written a line of code. No programming knowledge is required. Every step is explained from scratch.

Table of Contents

1. What Is Hubitat Dashboard?
2. Before You Begin — Hubitat Maker API Setup - Enable the Maker API App - Authorize Your Devices - Authorize Hub Variables - Find Your Connection Details
3. Web App — Windows Setup - Install Node.js - Get the Dashboard Files - Configure the Backend - Set a Control PIN - Start the Dashboard - Stop the Dashboard - Security Warning — Local Use Only
4. Web App — First-Time Use - Scan Your Devices - The "All Devices" Group
5. Web App — Building Your Dashboard - Creating a Custom Group - Adding Devices to a Group - Subgroups - Rearranging and Removing Tiles - Special Tiles
6. Web App — Settings
7. Android App — Installing the APK (Sideloading) - Step 1 — Get the APK File - Step 2 — Upload to Google Drive - Step 3 — Download to Your Phone - Step 4 — Install with the Files App - Updating the App Later - Build the APK Yourself (optional)
8. Android App — First-Time Setup
9. Android App — Building Your Dashboard - Creating a Custom Group - Adding Devices to a Group - Subgroups - Rearranging and Removing Tiles
10. Understanding Tile Types
11. GPS Tracking — Show Where Your Phones Are on a Map
12. Syncing Between Web and Android
13. Troubleshooting

---

1. What Is Hubitat Dashboard?

Hubitat Dashboard is a custom control panel for your **Hubitat Elevation** smart home hub. It shows your devices as tiles — small buttons or status displays — organized into groups that you define.

- The **web app** runs on a computer on your home network — **Windows, Linux, or macOS** — and is viewed in any browser.
- The **Android app** runs on your phone or tablet and connects to your hub directly.

Both apps let you: - See the live status of every device on your hub (switches, sensors, locks, lights, etc.) - Control devices by tapping their tiles - Organize devices into named groups and subgroups that make sense for your home, each with a custom icon - View hub variables (custom data values your hub tracks, like sunrise/sunset times) - View and control the Hub Security Manager (HSM) and hub mode - See where your phones are on a live **GPS map**, and generate AI **trip summaries** of a day's route (web app)

Neither app requires any paid subscription or cloud service. Everything runs on your local home network.

---

2. Before You Begin — Hubitat Maker API Setup

The Hubitat Maker API is a built-in feature of your Hubitat hub that lets third-party apps (like this dashboard) read and control your devices. **You must set this up before running the dashboard for the first time.**

2.1 Enable the Maker API App

1. Open a web browser and go to your Hubitat hub's web interface. - The address is usually something like http://192.168.1.x — your hub's IP address. - If you don't know the IP address, look at your router's connected devices list, or check the sticker on the back of the hub.
2. Click **Apps** in the left menu.
3. Click **Add Built-In App**.
4. Find **Maker API** in the list and click it.
5. The Maker API is now installed. Click **Maker API** in your Apps list to open its settings.

2.2 Authorize Your Devices

The Maker API will only expose the devices you explicitly allow.

1. In the Maker API settings page, find the section labeled **"Select which devices to allow access to"** (or similar wording).
2. Click in that field and select every device you want to see in the dashboard. - You can include all of them — it is safe to do so. - Any device you skip will not appear in the dashboard at all.
3. Check the box for **"Allow Access via Local Network"** — this is required for the dashboard to communicate with your hub.
4. If you want the Android app to also work away from home, check **"Allow Access via Cloud"** as well.
5. Click **Done** or **Update** to save.

2.3 Authorize Hub Variables

Hub variables are custom data values you have created in Hubitat (for example: Sunrise time, Sunset time, weather report text, custom flags). The dashboard can display these as tiles, but the Maker API must be told which variables it is allowed to share.

1. On the Maker API settings page, scroll down to find the section labeled **"Hub Variables"** or **"Select which Hub Variables to share"**.
2. Click in that field and select every variable you want to display in the dashboard. - If you have no hub variables, you can skip this step. - Common examples: Sunrise, Sunset, CivilDusk, AstronomicalDusk, WeatherReport
3. Click **Done** or **Update** to save.

> **Important:** If you add new hub variables later, you must return to the Maker API settings and authorize them before they will appear in the dashboard.

2.4 Find Your Connection Details

You need three pieces of information from the Maker API settings page. Write them down — you will enter them during setup.

What you need	Where to find it	Example
Hub IP Address	The address bar when you're on the hub's web interface, or listed on the Maker API page	192.168.1.42
Maker App ID	Shown on the Maker API settings page, labeled "App ID"	123
Access Token	Shown on the Maker API settings page, labeled "Access Token"	xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
Cloud Hub ID (Android only, for away-from-home access)	On the Maker API page under "Cloud Access"	606cf154-46af-4877-adb4-680b40e940c0

---

3. Web App — Windows Setup

3.1 Install Node.js

The web app requires a free program called **Node.js** to run. This is a one-time install.

1. Open your web browser and go to: **https://nodejs.org**
2. Click the large button that says **"LTS"** (recommended for most users) to download the installer.
3. Run the installer. Accept all the defaults — just keep clicking **Next** and then **Install**.
4. When it finishes, click **Finish**.

To verify it installed correctly: 1. Press the **Windows key + R**, type `cmd`, and press Enter to open a Command Prompt window. 2. Type the following and press Enter: `node --version` 3. You should see a version number like `v20.x.x`. If you do, Node.js is ready.

3.2 Get the Dashboard Files

You need the dashboard files on your computer.

Option A — Download a ZIP (easiest) 1. Go to the project's GitHub page in your browser. 2. Click the green **Code** button, then click **Download ZIP**. 3. When the download finishes, right-click the ZIP file and choose **Extract All**. 4. Choose a convenient location, for example `C:\HubitatDashboard`. 5. Click **Extract**.

Option B — Clone with Git (only if you already know what Git is)

```
git clone https://github.com/tibrown/HubitatDashboard.git C:\HubitatDashboard
```

All instructions below assume you placed the files in `C:\HubitatDashboard`. If you chose a different folder, substitute that path wherever you see `C:\HubitatDashboard`.

3.3 Configure the Backend

The backend is the part of the web app that talks to your hub. It needs to know your hub's address and access token.

1. Open **File Explorer** and navigate to `C:\HubitatDashboard\backend`.
2. You will see a file called `config.json.example`.
3. **Copy** that file (right-click -> Copy) and **paste** it in the same folder.
4. **Rename** the copy to `config.json` (right-click -> Rename).

5. Right-click config.json and open it with **Notepad** (or any text editor).
6. The file will look like this (this matches config.json.example): json { "hubIP": "192.168.1.xxx", "makerAppId": "123", "accessToken": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx", "backendPort": 3001, "pinHash": "$2b$10$...", "postUrl": "http://<THIS_SERVER_IP>:3001/api/webhook", "hubUsername": "", "hubPassword": "", "databaseUrl": "mysql://USER:PASS@localhost:3306/hubitat_dashboard", "gpsMap": { "csvUrl": "https://docs.google.com/spreadsheets/d/.../export?format=csv&gid=0" }, "home": { "label": "Home", "address": "123 Main St", "lat": 12.3456, "long": -78.9012, "radiusM": 250 }, "ollama": { "enabled": false, "baseUrl": "http://192.168.1.5:11434", "model": "qwen3.8:27b" }, "openrouter": { "enabled": false, "model": "deepseek/deepseek-v4-flash", "timeoutMs": 60000, "apiKey": "" } }
7. Replace the placeholder values with the details you wrote down in Section 2.4: - Change 192.168.1.xxx to your hub's actual IP address. - Change 123 to your Maker App ID. - Change the accessToken value to your Access Token. - Replace <THIS_SERVER_IP> in the postUrl with the IP address of **the Windows computer** that will run the dashboard (not the hub). You can find your PC's IP address by opening Command Prompt and typing ipconfig — look for the line that says "IPv4 Address".
8. pinHash — leave it alone for now, or **remove the whole line** to turn off PIN protection. To actually set a PIN for sensitive controls, see Section 3.3a — Set a Control PIN below.
9. The remaining fields are **all optional** and can be left exactly as-is on first setup: - hubUsername / hubPassword — only if your hub has Hub Security (login) enabled. Otherwise keep them blank. - databaseUrl — only if you want group config persisted to a MySQL database. If you're not using MySQL, you can **delete this line**. - gpsMap — URL of your public GPS Google Sheet. Needed only for the GPS map (see Section 11). - home — your home's address/coordinates, used to label GPS stops. Optional. - ollama / openrouter — AI providers for GPS trip summaries (see Section 11.7).
10. Save the file (Ctrl+S) and close Notepad.

Example of a filled-in config.json:

```
{
 "hubIP": "192.168.1.42",
 "makerAppId": "155",
 "accessToken": "a1b2c3d4-1234-5678-abcd-ef0123456789",
 "backendPort": 3001,
 "pinHash": "$2b$10$EjuBHH7i/mWZQbN70XMxt.2PW2e/ZovBq7imA/cRwDycOp95CZjVq",
 "postUrl": "http://192.168.1.100:3001/api/webhook",
 "hubUsername": "",
 "hubPassword": "",
 "databaseUrl": "mysql://bh:password@127.0.0.1:3306/hubitat_dashboard",
 "gpsMap": { "csvUrl": "https://docs.google.com/spreadsheets/d/1EGNmf9XvinmTE5EGZUOBjaoeU1HIvBLHlz33TN0KfcU/export?format=csv&gid=0" },
 "home": { "label": "Home", "address": "135 Cassady Street, Umatilla, FL", "lat": 28.9263, "long": -81.6646, "radiusM": 250 },
 "ollama": { "enabled": true, "baseUrl": "http://192.168.1.5:11434", "model": "qwen3.8:27b" },
 "openrouter": { "enabled": true, "model": "deepseek/deepseek-v4-flash", "timeoutMs": 60000, "apiKey": "" }
}
```

> The openrouter.apiKey value can be left blank if you instead set the OPENROUTER_API_KEY environment variable when starting the backend (recommended so the key is never in the file).

3.3a — Set a Control PIN (optional)

A 4-digit PIN can protect sensitive actions (arming/disarming security, locking, changing hub mode). To enable it, generate a bcrypt hash of your chosen PIN and paste it as pinHash: 1. The dashboard requires **Node.js** (installed in the next section) to generate the hash. In a Command Prompt/PowerShell window, in the dashboard folder, run: node -e "require('bcryptjs').hash('1234',10).then(console.log)" (Replace 1234 with the PIN you want.) 2. Copy the long $2... string it prints and paste it as the pinHash value in config.json. 3. Save the file and restart the dashboard.

To disable PIN protection, just remove the pinHash line entirely.

Set up the Webhook (Optional but recommended for live updates)

The webhook allows your hub to push device state changes to the dashboard instantly, so tiles update in real time without needing to refresh.

1. Go back to your Hubitat hub's web interface.
2. Open **Apps -> Maker API**.
3. Find the field labeled **"Post URL"** or **"Webhook URL"**.
4. Enter the value from the postUrl line in your config.json. For example: http://192.168.1.100:3001/api/webhook
5. Click **Update** or **Done**.

3.4 Install Dependencies and Start the Dashboard — Option A: npm run dev (Windows / macOS / Linux)

You only need to do the one-time "install" step once. After that, start the dashboard with npm run dev.

First-time install: 1. Press **Windows key + X** and choose **Windows PowerShell** (or **Terminal**). 2. Navigate to the dashboard folder by typing: powershell cd C:\HubitatDashboard and pressing Enter. 3. Type the following and press Enter. This downloads the software libraries the app needs (this installs both the backend and frontend in one go): powershell npm install This may take a minute or two. You will see a lot of text scroll by — that is normal. Wait for it to finish.

Starting the dashboard (every time): 1. Open PowerShell (as above) and navigate to the dashboard folder: powershell cd C:\HubitatDashboard 2. Type and press Enter: powershell npm run dev 3. This starts both the **backend** (http://localhost:3001) and the **frontend** dev server. You will see log lines for both in the same window. 4. Leave that window open. Open your web browser and go to: **http://localhost:5173** 5. The dashboard will appear.

> **Note:** It can take a few seconds for both services to come up. If you see the page immediately but the sidebar looks empty, wait a moment and refresh.

3.4b Start the Dashboard — Option B: start / stop shell scripts

The project also ships start and stop shell scripts that run both services in the background and write logs to .dashboard-backend.log / .dashboard-frontend.log. These work from **Git Bash** or **WSL** on Windows, and natively on macOS/Linux:

```
cd C:/HubitatDashboard       # or wherever you extracted the files
./start                      # starts backend (3001) + frontend (5173)
# open http://localhost:5173
./stop                       # stops both
```

If either port (3001 or 5173) is already in use, ./start will refuse to run — run ./stop first, or free the port.

3.4c Start the Dashboard — Option C: Docker (Raspberry Pi / NAS / server)

If you prefer containers, a docker-compose.yml is included (backend on :3001, frontend web server on port 80):

```
cp backend/config.json.example backend/config.json
# edit backend/config.json with your hub details
docker-compose up -d
# -> dashboard at http://<server-ip>   (frontend served on port 80)
```

> The Docker path serves the built web app on port 80. The npm run dev and ./start paths use the Vite dev server on port 5173. Use whichever fits your machine — the dashboard is identical.

3.5 Stop the Dashboard

- **If started with `npm run dev`:** press **Ctrl + C** in the same PowerShell window, then close it.
- **If started with `./start`:** run `./stop` in the dashboard folder, or close both service windows.
- **If started with Docker:** run `docker-compose down` in the dashboard folder.

3.6 Security Warning — Local Use Only

> **This dashboard is designed to run ONLY on your home network.**

> The web app has **no login, no password, and no encryption**. Anyone on your local network who knows the address (`http://localhost:5173` or your PC's IP address on port 5173) can see and control your devices.

> **Do not expose this dashboard to the internet.** Do not port-forward or publish this address outside your home. It is intended only for personal use on your private Wi-Fi network.

---

4. Web App — First-Time Use

4.1 Scan Your Devices

The first time you open the dashboard, it has no groups configured. The first thing you should do is scan your hub to load all of your devices.

1. Open the dashboard in your browser (`http://localhost:5173`).
2. Look at the left sidebar. You will see a settings or menu icon — click it to open the sidebar.
3. Click the **Scan** or **Refresh Devices** button. The app will contact your hub and load every device you authorized in the Maker API.

4.2 The "All Devices" Group

After scanning, the dashboard creates one built-in group called **"All Devices"** (sometimes shown as **"Other"**). This group automatically contains every device that was found on your hub but has not yet been placed in a custom group.

Think of it as a holding area — a complete inventory of everything the hub knows about. From here, you will gradually move devices into organized groups that make sense for your home.

> **Tip:** The "All Devices" group can get large. That is okay. As you create your own groups and add devices to them, those devices will be removed from "All Devices" automatically.

---

5. Web App — Building Your Dashboard

5.1 Creating a Custom Group

Groups are pages in the sidebar that hold related tiles. You might make a group called "Living Room", "Garage", "Security", etc.

1. In the sidebar, look for a **"+"** or **"New Group"** button and click it.
2. A dialog box will appear. Type a name for your group (for example: `Living Room`).

3. Choose an icon from the icon grid by clicking on one. The icon will appear next to your group's name in the sidebar.
4. Click **Create Group**.
5. Your new empty group will appear in the sidebar. Click it to open it.

5.2 Adding Devices to a Group

Once you're viewing a group, you can add device tiles to it.

1. Open the group you want to add devices to by clicking its name in the sidebar.
2. Look for an **Edit** mode button (often a pencil icon or "Edit" label) in the top-right area of the page. Click it to enter edit mode. - In edit mode, you will see buttons to add, remove, and rearrange tiles.
3. Click the **"Add Device"** button (usually a **+** symbol or labeled "Add Tile").
4. A picker panel will appear showing a searchable list of all your devices. - At the top of the list you will see **Special Tiles** — these include Hub Mode, Security System (HSM), and time-based tiles like Sunrise and Sunset. See Section 5.5 for details. - Below that is the full list of your devices, sorted alphabetically.
5. You can type in the **Search** box to filter the list. - Try searching by room ("kitchen"), device type ("motion"), or device name ("front door").
6. Click on any device to add it to the group. The picker will close and the tile will appear.
7. Repeat for as many devices as you want to add to this group.
8. When finished, click the **Edit** button again (or a **Done** button if shown) to exit edit mode.

> **A device can appear in more than one group.** For example, "Front Door Lock" might appear in both a "Locks" group and a "Front Entryway" group. There is no restriction.

5.3 Subgroups

A subgroup is a group that lives inside another group. This is useful for organizing large groups into sections. For example:

- **Security** (main group)
- **Motion Sensors** (subgroup)
- **Door & Window Contacts** (subgroup)
- **Cameras** (subgroup)

To create a subgroup (web): 1. Follow the steps in Section 5.1 to create a new group. 2. After typing the group name and choosing an icon, look for a **"Parent Group"** dropdown menu in the dialog. 3. Click the dropdown and choose the group that should contain this new subgroup. 4. Click **Create Group**.

The new subgroup will appear nested under its parent in the sidebar.

> **Note:** Subgroups work the same as regular groups — you can add devices to them, rename them, etc. The only difference is how they are displayed in the navigation.

5.4 Rearranging and Removing Tiles

While in **edit mode**:

- **To move a tile:** Click and drag it to a new position within the group.
- **To remove a tile:** Look for a small **X** or **Remove** button that appears on each tile in edit mode. Click it to remove that tile from the group.
- Removing a tile from a group does not delete the device from the hub — it only removes it from this group's view.

5.5 Special Tiles

In addition to regular devices, you can add several **special tiles** to any group:

Tile	What it shows
Hub Mode	The current mode of your Hubitat hub (e.g., Home, Away, Night). Tap to change the mode.
Hub Security Manager (HSM)	The status of the built-in alarm/security system. Tap to arm or disarm.
Sunrise	The time of today's sunrise (from a hub variable).
Sunset	The time of today's sunset.
Civil Dusk	The time of civil dusk (slightly after sunset).
Full Dark	The time of astronomical dusk (full darkness).
Weather Report	A text summary if you have a weather hub variable set up.

These appear at the top of the device picker under "Special Tiles".

> **Hub variables must be authorized in Maker API** (see Section 2.3) for these tiles to display data. If a hub variable tile shows "—" or blank, check that the variable is authorized and spelled correctly.

---

6. Web App — Settings

Click the **Settings** (gear icon) in the sidebar to open the Settings panel.

Setting	What it does
Idle Auto-Refresh	Automatically reloads the dashboard after N minutes of no activity. Set to 0 to disable. Useful if the dashboard is shown on a wall-mounted screen.
Hub File Manager Credentials	Only needed if you have **Hub Security** (login/password) enabled on your Hubitat hub. Used for the Push/Pull config sync feature. Leave blank if you haven't enabled hub security.

---

7. Android App — Installing the APK (Sideloading)

The Hubitat Dashboard Android app is not on the Google Play Store. Instead, you install it by **sideloading** — manually transferring the APK file to your phone and installing it. This is safe and common for personal apps.

7.1 Step 1 — Get the APK File

1. The APK file is named something like `hubitat-dashboard-debug.apk` or `app-debug.apk`.
2. It will be provided to you by whoever built the app, or you can find it in the project files under `android/app/build/outputs/apk/debug/`.
3. Save it somewhere easy to find on your Windows PC (for example, your Desktop).

7.2 Step 2 — Upload to Google Drive

1. On your Windows PC, open a web browser and go to **https://drive.google.com**.
2. Sign in with your Google account.
3. Click **+ New** in the top-left, then choose **File upload**.
4. Browse to the APK file (e.g., your Desktop) and click **Open**.
5. Wait for the upload to finish. The APK will now appear in your Google Drive.

7.3 Step 3 — Download to Your Phone

1. On your **Android phone**, open the **Google Drive** app. - If you don't have it, download it from the Play Store — it's free.
2. Find the APK file you just uploaded.
3. Tap the **three dots (...)** next to the APK filename.
4. Tap **Download**.
5. The file will be saved to your phone's Downloads folder.

7.4 Step 4 — Install with the Files App

Android does not allow installing apps from unknown sources by default. You will be prompted to allow it once.

1. Open the **Files** app on your phone. - On most Android phones this is called **Files by Google**, **My Files**, or just **Files**. - If you can't find it, search for "Files" in your app drawer.
2. Navigate to **Downloads** (or the folder where the APK was saved).
3. Tap the APK file (e.g., app-debug.apk).
4. A prompt will appear asking if you want to install the app. - If a dialog says **"For your security, your phone is not allowed to install unknown apps from this source"**, tap **Settings** within that dialog, then enable the toggle that says **"Allow from this source"**, then press the back button.
5. Tap **Install**.
6. Tap **Done** when the installation finishes.
7. The **Hubitat Dashboard** app is now installed. You can find it in your app drawer.

> **Note:** The "install unknown apps" permission applies only to the Files app, not to every app on your phone. This is safe.

7.5 Updating the App Later

When a new version of the APK is available:

1. Upload the new APK to Google Drive (it can overwrite the old one).
2. Download it to your phone.
3. Tap the APK in the Files app and tap **Install** — it will update the existing app.
4. All your group configuration and settings are preserved.

7.6 Build the APK Yourself (optional — for the developer-minded)

If you don't have a prebuilt APK — or you want the very latest build — you can build it yourself using the same process the maintainers use.

Prerequisites (one-time): - **JDK 17** (Java 17). Download from adoptium.net (Temurin 17) or install via your package manager. - **Android SDK** command-line tools (or Android Studio). The build looks for the Android SDK at /opt/android-sdk by default; if yours is elsewhere, set the ANDROID_HOME (or ANDROID_SDK_ROOT) environment variable to its location.

Steps: 1. Open a terminal — **Git Bash** or **WSL** on Windows, **Terminal** on macOS/Linux — and go into the android folder: bash cd C:/HubitatDashboard/android # Git Bash: cd /c/HubitatDashboard/android 2. Build the debug APK: bash bash gradlew assembleDebug

> Use bash gradlew — the gradlew file here is a plain shell script, not the usual binary wrapper, so ./gradlew may not work even after chmod +x. 3. When it finishes, the APK is at: android/app/build/outputs/apk/debug/app-debug.apk 4. **Every successful build is assigned a new version automatically** (1.0.1, 1.0.2, ...) so your phone sees each rebuild as an upgrade. Install the newest APK to get the latest. 5. Transfer the APK to your phone (Steps 2–4 above), or if the phone is connected via USB with developer tools, install it with adb: bash adb install -r android/app/build/outputs/apk/debug/app-debug.apk

> **Tip — if a fresh build won't replace an installed one:** the counter restarts at 1.0.1 if the build-number cache is ever deleted. Android treats an equal versionCode as a reinstall (data preserved) and only forces an upgrade when the number rises. If you need to force an upgrade over an already-installed build, bump the version first.

---

8. Android App — First-Time Setup

When you open the Hubitat Dashboard app for the first time, it will take you directly to the **Settings screen** because no hub has been configured yet.

Fill in the following fields:

Field	What to enter	Where to find it
Local Hub IP	Your Hubitat hub's IP address on your home network	e.g., 192.168.1.42 — written on hub or in router's device list
Maker App ID	The App ID from the Maker API settings page	e.g., 155
Access Token	The Access Token from the Maker API settings page	The long string of letters, numbers, and dashes
Cloud Hub ID	(Optional) Your hub's cloud ID, for away-from-home access	Found on Maker API page under "Cloud Access"
Control PIN (optional)	A 4-digit PIN that protects security, hub-mode, and lock actions	Enter it here once; you'll be asked for it when you arm/disarm security, change mode, or unlock. To change or clear it later, return here.
GPS Apps Script URL (optional)	Where the app posts your location for the GPS map	Your own Google Apps Script endpoint — see Section 11
GPS Device Name (optional)	A name that tags your location in the log (e.g. "My Phone")	Whatever you like — it appears as the device filter on the GPS map
Enable Perimeter Watch (optional)	Alerts (with sound) when this phone comes within the radius of the other phone	GPS Tracker screen — see 11.5. Home phone must be tracking to the same sheet

Connection Mode: - **Local** — Connect to your hub only on your home Wi-Fi. Use this if you only want to control your home when you're at home. - **Cloud** — Connect through Hubitat's cloud service. Works anywhere with internet, but slightly slower. - **Auto** — Try local first; fall back to cloud if local is unavailable. Recommended.

After entering your details:

1. Tap **Test Connection** to verify the app can reach your hub. You should see a success message.
2. Tap **Save** to save your settings.
3. The app will load your devices and take you to the dashboard.

If the connection test fails: - Double-check your Hub IP and Access Token. - Make sure your phone is on the same Wi-Fi network as your hub (for Local mode). - Make sure the Maker API is enabled and the Access Token matches exactly.

---

9. Android App — Building Your Dashboard

The Android app works similarly to the web app. Your devices are organized into groups that you create.

9.1 Creating a Custom Group

1. Tap the **navigation drawer** (hamburger menu) or swipe from the left edge.
2. Tap the **"+"** icon or **"New Group"** button.
3. A panel will appear from the bottom of the screen.
4. Type a name for your group (e.g., Backyard).
5. Choose an **icon** by tapping one in the icon grid.
6. Optionally set a **Parent Group** (see Section 9.3).
7. Tap **Create**.

9.2 Adding Devices to a Group

1. Navigate to the group you want to add devices to (tap its name in the menu).
2. Tap the **Edit** button (pencil icon) in the top-right corner.
3. Tap the **"+"** or **"Add"** button that appears.
4. A device picker panel will slide up from the bottom.
5. You will see two sections: - **Special Tiles** — Hub Mode, Security System, time variables like Sunrise/Sunset - **Devices** — all devices loaded from your hub, sorted alphabetically
6. Use the **Search** field to filter devices by name or type.
7. Tap a device or special tile to add it to the group.
8. Repeat until you've added everything you want.
9. Tap the **Edit** button again (or tap outside the panel) to exit edit mode.

9.3 Subgroups

A subgroup is a group that appears nested inside another group. This helps organize large dashboards.

Example structure: - Outdoor (parent group) - Backyard (subgroup — add outdoor motion sensors here) - Front Yard (subgroup — add front door and gate sensors here) - Garden (subgroup — add irrigation and outdoor lights here)

To create a subgroup: 1. Follow the steps in Section 9.1 to open the "Create Group" panel. 2. At the bottom, find the **Parent Group** dropdown. 3. Tap the dropdown and choose which existing group should be the parent. 4. Tap **Create**.

The new subgroup will appear under its parent in the navigation menu, indented to show the relationship.

> **Tip:** Subgroups are perfect for large homes. For example, you could have a "Upstairs" group with subgroups for each bedroom, or a "Security" group with subgroups for "Perimeter", "Motion", and "Cameras".

9.4 Rearranging and Removing Tiles

While in **edit mode** (pencil icon active):

- **To move a tile:** Press and hold a tile, then drag it to a new position.
- **To remove a tile:** Tap the **X** button that appears in the corner of each tile.

---

10. Understanding Tile Types

When you add a device, the app automatically picks the best tile type for it based on its capabilities. Here is what each tile type looks like and does:

Tile Type	What it shows	What you can do
Switch	On/Off status with colored indicator	Tap to toggle on or off
Dimmer	On/Off plus brightness level	Tap to toggle; slider to set brightness
RGBW / Color Light	Color swatch plus brightness	Tap for full color and brightness control
Contact	Open/Closed status (door or window)	Display only — shows if open or closed
Motion	Active/Inactive status	Display only — shows if motion is detected
Presence	Present/Away	Display only — shows if a person or device is home
Temperature	Current temperature reading	Display only
Lock	Locked/Unlocked	Tap to lock or unlock (may require PIN)
Button	Button device	Tap to push the button
Power Meter	Current wattage	Display only
Connector	Virtual on/off switch	Tap to toggle — used for virtual switches that trigger automations
Waterison	A water/valve tile with a live countdown while a timer is running (from the WaterTimeout hub variable)	Display only — shows the remaining time and updates live
Multi-Device	One tile that packs several devices together, each with its own cell and toggle	Tap each cell to control that device in place
Ring Detection	Ring camera motion/ring event	Display only — shows last detection event
Hub Mode	Current hub mode (Home, Away, Night, etc.)	Tap to change mode
HSM	Security system arm status	Tap to arm/disarm (may require PIN)
Hub Variable	Custom text or value from hub	Display only

> The app makes a best guess at the tile type based on the device's capabilities. You can change it manually if the guess is wrong by using the tile type picker in edit mode.

---

11. GPS Tracking — Show Where Your Phones Are on a Map

The dashboard can show a live map of where your Android phones are. This is a three-part pipeline, and each part must be in place for it to work:

1. **Each Android phone** runs the dashboard's built-in GPS tracker, which posts its location to a **Google Sheet**.
2. The **web dashboard** (and the Android app) read that sheet once it's **published to the web**.
3. The map is drawn from the sheet's rows. Optionally, an **AI trip summary** narrates a day's route.

```
Android phone (GPS tracker) -> Google Apps Script -> Google Sheet -> published CSV -> dashboard map
```

11.1 One-time: create the Google Sheet

1. Create a new Google Sheet (sheets.new).
2. Put a header row in the **first row** with a timestamp column, a latitude column, and a longitude column: - Latitude: header lat (the app also accepts latitude). - Longitude: header long (also accepts longitude, lon, or lng). - (Optional) a name column tags each point with a phone name — this is what drives the **phone filter** dropdown on the map. If you omit it, every point shows under one device.
3. **Publish it to the web:** File -> **Share** -> **Publish to web** -> click **Publish**. (Set it to "Anyone with the link" so the dashboard and phones can both read it.)
4. Copy the **CSV export URL**: it's the spreadsheet link with /export?format=csv&gid=0 appended, e.g.: https://docs.google.com/spreadsheets/d/1EGNmf9XvinmTE5EGZUOBjaoeU1HIvBLHlz33TN0KfcU/export?format=csv&gid=0

11.2 One-time: create the Google Apps Script "receiver"

Phones need a way to write to the sheet. The usual approach is a small Google Apps Script web app:

1. In the same spreadsheet, open **Extensions -> Apps Script**.
2. Add a doPost(e) function that parses the incoming lat/long/timestamp/name and appends a row to the sheet.
3. **Deploy -> New deployment -> Web app**, set access to **"Anyone"** (or your domain), and copy the /exec URL.
4. That /exec URL is what you'll put in the Android app as **GPS Apps Script URL**.

> The dashboard itself only ever reads the sheet (via the CSV export URL). The Apps Script receiver is the write side that the phones talk to.

11.3 Configure the web backend

In backend/config.json (see Section 3.3):

- **gpsMap.csvUrl** — paste your sheet's CSV export URL from 11.1.
- **home** (optional) — your home's label, address, lat/long, and radiusM. Any GPS stop within that radius is labeled **"Home"** and is excluded from business lookup in trip summaries.

11.4 Turn on tracking on each Android phone

On each phone that should be tracked, in the app's **Settings**:

1. **Grant background location permission.** On Android, the app needs location access **"Allow all the time"** (Settings -> Privacy -> Location -> the location toggle -> Allow all the time). If it only has "While using the app", the tracker goes silent in the background.
2. Set **GPS Apps Script URL** to your Apps Script /exec URL (from 11.2).
3. Set **GPS Device Name** (e.g. My Phone).

4. Save, then tap the **GPS test / "Test Now"** button to send a single point immediately and confirm the whole pipeline works.

> New phones are extra-prone to this: OEM battery optimizations sometimes kill background location services. If a phone simply stops appearing, re-check the permission above, then test.

11.5 Perimeter Watch (approach alert)

Perimeter Watch alerts you — a notification with sound — when this phone (the traveler) comes within a set distance of the other phone (the base, e.g. the phone that stays home).

1. Both phones must be tracking to the **same Google Sheet** (see 11.1–11.4). The traveling phone runs the app with Perimeter Watch checked; the base phone just needs its normal tracking running.
2. In the app's **GPS Tracker** screen, check **Enable Perimeter Watch**.
3. **Other phone device name** defaults to `Pixel 9 Pro` — it must match the base phone's **GPS Device Name** exactly as it appears in the sheet's `device` column.
4. **Perimeter radius** defaults to `1.0` miles. You can change it anytime; new value applies on the next check.

How it behaves:

- The app compares its location against the base phone's **latest** fix in the sheet, re-checking at most every ~30 seconds while tracking is active.
- You are alerted **once per approach** — when crossing from outside to inside the radius. Leave range and come back and it alerts again.
- If the base phone hasn't posted a fix for **20+ minutes**, checks pause so a stale last-known spot can't false-trigger.
- The notification uses a high-importance channel and **plays your notification sound**, unless the phone is on **Do Not Disturb** or at zero volume (platform limit).

11.6 View the map

- **Web:** open the **GPS Track** page (in the sidebar). The map defaults to **today** and shows only phones that have points in the selected range. Use the controls to:
- filter by **phone** (only phones with points in the date range are listed),
- change the **date range**,
- toggle **fullscreen**, and
- (optionally) enable **road distance** via OSRM for driving-distance legs.
- **Android:** open **GPS** from the main navigation menu for the same map on your phone.

11.7 GPS AI Trip Summary (the "Report" button)

On the GPS Track page, the **Report** (green) button turns the currently-filtered route into a short, human-readable trip summary — naming the businesses and addresses you stopped at, how long you stayed, and travel time between stops.

- If both AI providers are configured, a dropdown lets you pick the backend: | Provider | Where it runs | Privacy | |---|---|---| | **Local AI (Ollama)** | on your LAN | coordinates never leave your network | | **Cloud (OpenRouter)** | openrouter.ai | sends coordinates/timestamps to the cloud |
- The providers are configured in `backend/config.json` as the **`ollama`** and **`openrouter`** blocks (both optional; if neither is present the Report feature is hidden).
- Stops are detected automatically server-side; each stop is named via **OpenStreetMap / Overpass** (nearest business or street address). With the `home` block set, trips stopping at home are labeled "Home — " instead of a nearby business.

11.8 GPS Troubleshooting

Symptom	Likely cause / fix
No points on the map	Phone lacks background location permission — set **"Allow all the time"** (11.4). Then use **Test Now** to confirm.
Sheet isn't growing	GPS Apps Script URL is wrong or the script isn't deployed as a web app with append logic. Check the script's Execution log.
Map empty for a day that should have points	Date range isn't covering it (it defaults to today), or the phone filter is set to another device.
"Phone not in the dropdown"	That phone has no points **in the selected date range** — widen the range.
Tracker stops after a phone restart/app update	The tracker re-asserts itself on launch **only when GPS tracking is enabled and configured** — re-open Settings to make sure the URL + name are saved.
Perimeter Watch never alerts	Check: base phone is actively tracking to the **same sheet** (its fix must be under 20 min old), its **device name** matches exactly, and the phone isn't on **Do Not Disturb**/zero volume (sound won't play, though the notification still shows).

---

12. Syncing Between Web and Android

Your group and tile configuration can be shared between the web app and Android app.

Exporting from the Web App

1. On the web app, open the sidebar and find the export/import option.
2. Click **Export Config**. A JSON file will be downloaded to your PC.

Importing into Android

1. Transfer the JSON file to your phone (email it to yourself, use Google Drive, etc.).
2. In the Android app, go to **Settings**.
3. Tap **Import Config**.
4. Browse to the JSON file and select it.
5. Confirm when prompted.

Push / Pull via Hub File Manager

Both apps support pushing and pulling the config file directly through the Hubitat hub's built-in file manager. This allows seamless sync without manually transferring files.

1. In the web app: click **Push to Hub** to save the current config to your hub.
2. In the Android app: tap **Pull from Hub** to download that config.

> This feature requires your hub's IP/login credentials if Hub Security is enabled on the hub. Enter those in the Hub File Manager Credentials section of Settings.

---

13. Troubleshooting

"I can't see any devices after scanning"

- Confirm the Maker API app is installed and enabled on your hub.
- Check that you selected devices in the Maker API's device list (see Section 2.2).
- Verify the Hub IP, Maker App ID, and Access Token are entered correctly.
- Make sure your computer or phone is on the same network as the hub.

"Tiles are not updating in real time"

- The webhook may not be configured. Set up the Post URL in the Maker API settings (see Section 3.3).
- The dashboard will still update by polling the hub periodically even without the webhook — it just won't be instant.

"A hub variable tile shows nothing"

- Make sure the variable is added to the Maker API's allowed variable list (see Section 2.3).
- The variable name must be spelled exactly as it appears in Hubitat (it is case-sensitive).

"I can't install the APK on my Android phone"

- Make sure you enabled "Allow from this source" for the Files app when prompted (see Section 7.4).
- If the APK fails to install with "App not installed", try restarting your phone and trying again.
- If you get "Parse error", the APK file may be corrupted — try downloading it again.

"The web app won't start on Windows"

- Make sure Node.js is installed (type `node --version` in PowerShell).
- Make sure you ran `npm install` in the `C:\HubitatDashboard` folder at least once.
- Check that `backend/config.json` exists and has your hub IP and access token filled in.
- Make sure nothing else is using ports 3001 or 5173. Run `./stop` first if you're unsure (or stop any leftover `npm run dev` with Ctrl+C).

"The Android app says 'Connection failed' or 'Unable to reach hub'"

- Make sure your phone is on the same Wi-Fi network as your hub (for Local mode).
- Double-check the Hub IP and Access Token.
- Try tapping **Test Connection** in Settings.
- If you want to use the app away from home, make sure "Allow Access via Cloud" is enabled in the Maker API and enter your Cloud Hub ID in Settings.

"Android flags my install as 'unrecognized developer' / Play Protect blocks it"

Debug APKs are signed with a local debug key that Google Play Protect has never seen, so it can warn even when the file installs fine elsewhere. - Easiest personal fix: **Play Store -> Profile -> Play Protect -> Settings -> turn OFF "Scan apps with Play Protect"**, then install. - Or, on the warning dialog tap **More details -> Install anyway** (this whitelists the signing key for future updates too).

"Mode / Security / hub-variables tiles are empty on the Android app, especially on Cloud"

On a fresh **Cloud** connection the app must fetch the mode, HSM, and hub-variable values separately — they don't arrive with the device list. Open the group page once on the Cloud connection (or tap refresh in the app) so those special tiles load. If they stay empty, re-check that those hub variables are authorized in the Maker API (Section 2.3).

"The GPS map is empty / a phone is missing" (Android)

See Section 11.7 — the usual culprit is background location permission not set to **"Allow all the time"**, or the date/phone filter hiding the points.

This guide covers everything needed to set up and use Hubitat Dashboard from scratch. If you run into something not covered here, the most common cause is a mismatch between the Maker API settings on the hub and what is entered in the app's configuration.